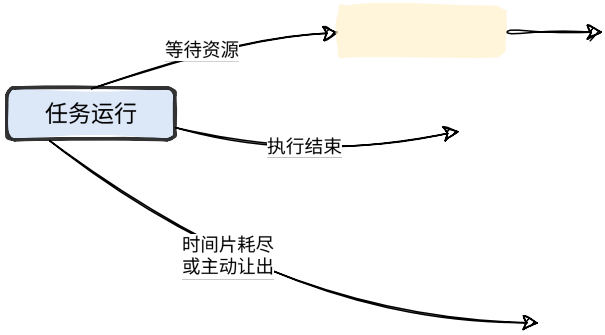
第 3 章 调度与同步机制

任　等待事件的　程看起来直接：　　条件、登　　醒入口、　入阻塞，事件到了再恢　。但如果通知恰好在“条件　　完、Waker　　”

Waker 之　有空隙，Futex 通知与入　之　存在同　的　争。

moss 沿用 StarryOS 的 RR



内容	StarryOS 基	moss 的增量
任　　度	任　状　、RR　　CPU	沿用　度框架，修正　和性位　解析
阻塞　醒	Future、Waker 和等待　列	PollSet
定　　理	基　休眠和　程定　器	区分相　与　　，使用 TimerKey
Futex	WAIT、WAKE、位掩　和重新排	加入独立通知状　，修正超　　和退出清理
子　程等待	退出事件与基　等待	将状　　化接入 Waker，并在登　后条件

表 3-1

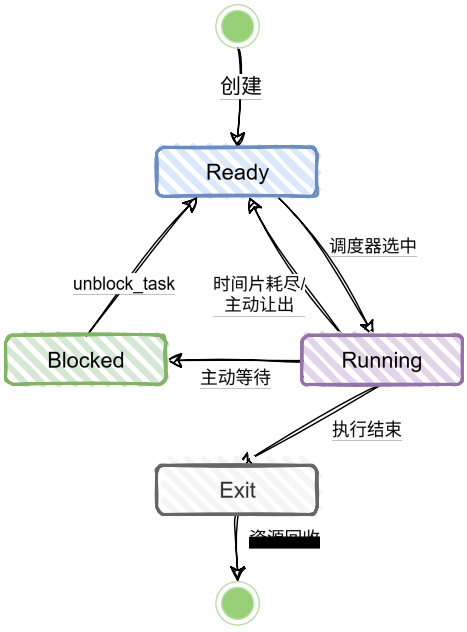
CPU 和重新　得 CPU 的　程。其中最　　理的是通知与状　可能同　　生：任　　完条件、尚未登　Waker

moss 在阻塞　醒机制上的改　。

3.2 任务调度机制

3.2.1 任务状态与调度流程

StarryOS 的任　　度　　四　状　展　：`Running`（正在 CPU 上　行）、`Ready`（可　行，在　列中等待）、`Blocked`（等待事件）、`Exited`（已　束）。新任　以 `Ready`　`Running`。主　　出 CPU 的任　回到 `Ready`；等待　源的任　　入 `Blocked`，由 `unblock_task` 恢　　`Ready`。

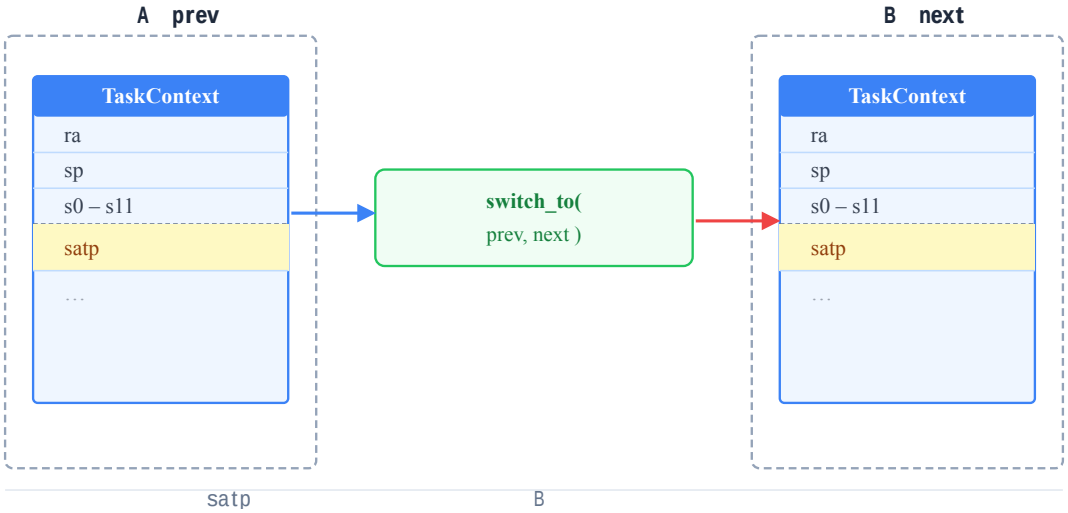


3-2　TaskState 的状　　系

任　按 RR 策略　　使用 CPU，默　　片　5 个　　滴答。　　中断到来　更新　　片，数　零　先　置待　占　志，等代　离　禁止　占的　界区后再　行重新　度。　　可以避免在持　或中断　理　切　任　。

→　　→　　→ 离　　界区 → 重新　度

多核　境下，每个 CPU 都有自己的就　　列。　度器从　　下一个任　，　　片用完后任　回到　尾，CPU　　idle 任　接管 CPU。上下文切　，当前 CPU 的寄存器　被保存到旧任　的 `TaskContext` 中，再从新任　的 `TaskContext` 恢　。



3-3　TaskContext 布局与 switch_to 的　　方向

moss 在　建任　　把　　程的　表根写入 `TaskContext`，保　切　后使用正的地址空　。用　　寄存器　保存在 `UserContext` 中。整个流程可以串成：

→ 到　安全　度位置 →　　→ 保存旧　　→ 恢　新

3.2.2　CPU 亲和性修正

每个任　保存一份 CPU　　CPU 上　行。moss 修正了用　位解析　程：按　　配置的 CPU 数量　取内容，把置　1 的位写入任　掩　。

```
// 解析用户传入的亲和性位图
for cpu_id in 0..axconfig::plat::CPU_NUM {
    if user_mask[cpu_id / 8] & (1 << (cpu_id % 8)) != 0 {
        cpu_mask.set(cpu_id, true);
    }
}

// 保存新掩码
let curr = current().clone();
curr.set_cpumask(cpu_mask);

// 当前 CPU 已不在范围内时，迁移任务
if !cpu_mask.get(axhal::percpu::this_cpu_id()) {
    let migration_task = TaskInner::new(
```

```
        move || migrate_entry(curr),
        "migration-task".into(),
        4096,
    ).into_arc();
    current_run_queue().migrate_current(migration_task);
}
```

代　3-1　CPU

新掩　保存后　要　　任　当前所在 CPU，如果已被排除，会将任　　移到另一个允　的行　列。用　程序通　`sched_getaffinity`　　`sched_setaffinity` 修改范　。

3.3　阻塞与唤醒机制的核心改进

moss 的重点改　。StarryOS 已　有了基本的 Future/Waker 模型，但在通知与阻塞交　　容易出　　。moss 的改　　一个核心思路展　：　　。

3.3.1　事件等待的基本流程

`Blocked`，并不知道任　在等什么。事件　象和等待者之　需要一套通知　　。moss 沿用 Future 表示“尚未完成、之后　要　　的操作”，用 Waker　“条件可能　化　如何通知当前任　”。

以　取管道　例，任　先　　冲区。已有数据就直接　取；没有数据　把自己的 Waker 登到管道中，再由 Future 返回“　　”。另一个任　写入数据后，管道　用保存的 Waker，通知　任　　。

```
                    → 保存 Waker → 再次　　→ 等待通知

事件　生 →　　Waker → Future 再次　行 → 重新　　条件
```

3-4　事件登　与通知的基本流程

Waker 后的第二次条件　　。　如数据恰好在第一次　　束后到　，写入方　　不到等待者，　　用 Waker。登　完成后立　　冲区，　任　　能看到已到　的数据，不会　　次通知　一直等待。

到通知后　要重新　　条件。Waker 只表示“　　可能有　化”，并不保　　源一定　　当前任　。例如多个　任　同　被通知　，管道中的数据可能已　被其中一个任　取走，其　任　需　　等待。

3.3.2　PollSet:　锁外唤醒

StarryOS 的 `PollSet` 管理一　Waker，但　醒　可能在持　　下　用 `wake()`。moss 的改直接：加个　子　志　　断有没有等待者，　后把 Waker 取出来到　　再　用。

```
// vendor/axpoll/src/lib.rs
pub struct PollSet {
    inner: Lazy<Mutex<Inner>>,
    has_waiters: AtomicBool,  // 快速判断有没有等待者
}
```

```rust
pub fn wake(&self) -> usize {
    if !self.has_waiters.load(Ordering::Acquire) {
        return 0;  // 没人等，直接返回
    }
    let (wakers, len) = self.inner.lock().take_wakers();  // 取出
    self.has_waiters.store(false, Ordering::Release);
    drop(guard);  // 释放锁
    for waker in wakers.iter().take(len) {
        unsafe { waker.assume_init_read() }.wake();  // 锁外唤醒
    }
    len
}
```

代　3-2　PollSet 的　　醒

3.3.3　block_on: 用标记补窗口

Future 返回 Pending 和通知到　可能　　同　　生。如果通知先到，但任　　没　入 Blocked 状　，通知就　　了。

moss 的解　　是加一个 woke

```rust
// vendor/axtask/src/future/mod.rs
struct AxWaker {
    task: WeakAxTaskRef,
    woke: SpinNoIrq<bool>,  // 关键：记录通知有没有到
}

// 通知来的时候：
impl Wake for AxWaker {
    fn wake_by_ref(self: &Arc<Self>) {
        if let Some(task) = self.task.upgrade() {
            *self.woke.lock() = true;  // 先标记
            select_run_queue(&task).unblock_task(task, false);  // 再唤醒
        }
    }
}

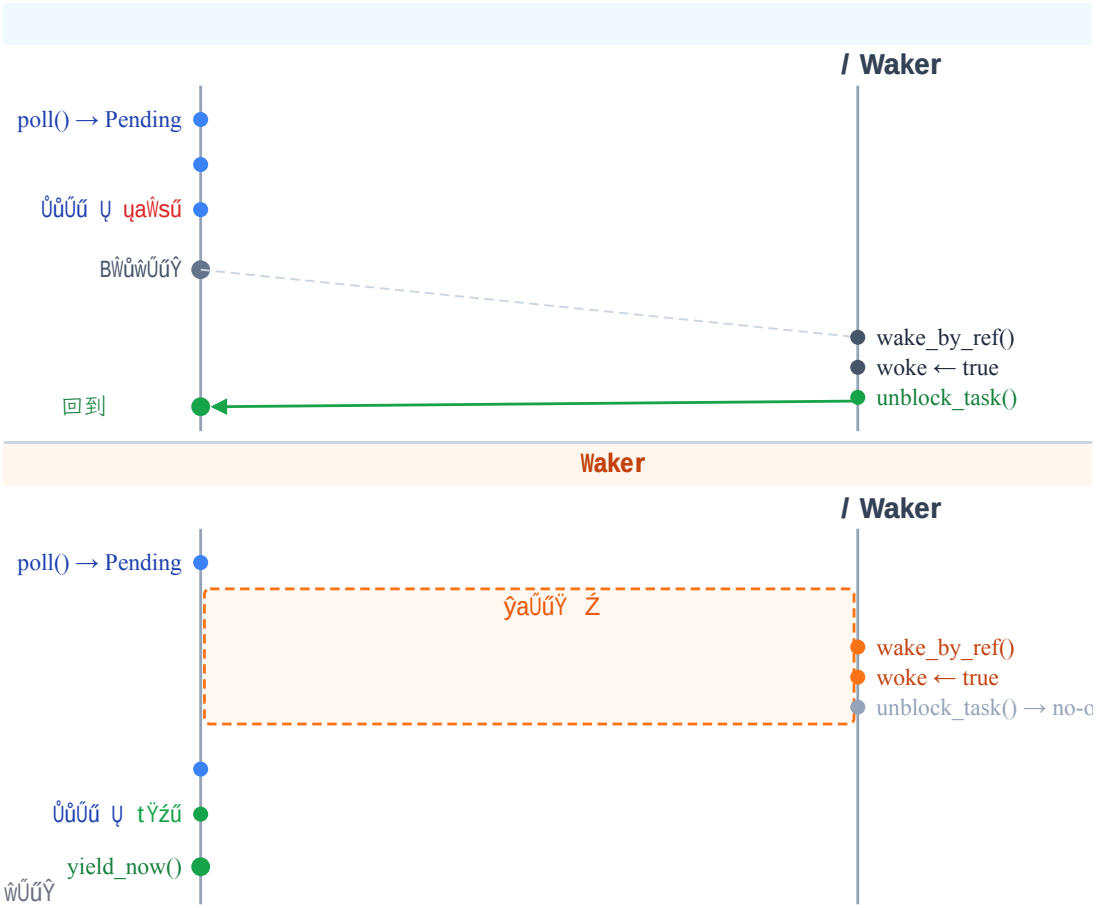
// 准备阻塞的时候：
match fut.as_mut().poll(&mut cx) {
    Poll::Pending => {
        let woke = axwaker.woke.lock();
        if !*woke {
            rq.blocked_resched(woke);  // 没通知，安全阻塞
        } else {
            drop(woke);
            crate::yield_now();  // 通知先到了，只让出 CPU
```

```
        }
    }
    Poll::Ready(output) => break output,
}
```

代 3-3 woke

ū

Ŭ 3-5 展示了 ŭ 条路 的 序 。



3-5 ŭ ż Ž Ŭ ž vs 通知先到

3.3.4 定时器: 用序号去重

StarryOS 的定 器用截止 ，但定 器重新 置后，旧事件 留在 列里。moss 的是 每个事件加个 增的序号：

```
struct TimerKey {
    deadline: Duration,  // 什么时候触发
    seq: u64,            // 第几个事件
}

impl Ord for TimerKey {
    fn cmp(&self, other: &Self) -> Ordering {
        // 先按截止时间排序，再按序号排序
        self.deadline.cmp(&other.deadline)
```

```
            .then_with(|| self.seq.cmp(&other.seq))
    }
}

struct AlarmRuntime {
    next_seq: u64,                          // 生成序号
    timers: BTreeMap<TimerKey, Entry>,  // 按时间顺序存
}
```

代 3-4 TimerKey = 截止 + 序号

重新 置定 器 ſ除旧 ，后 任 取出事件后 序号 配 。 旧事件就被自 了。同一 可能有多个定 器到 ，seq 使每个事件都有独立的 。新增更 的定器 ，系ƀ 会通知后 任 重新 下一次等待 。

每个 程 有一个 管理 象 TimeManager，用来分Ɓ 用 和内核 行 。定 器 ITIMER_REAL 按 SIGALRM；用 定 器 ITIMER_VIRTUAL 只ƀ 用；性能分析定 器 ITIMER_PROF 同 ƀ 用 与内核 。 定 器到 后重新Ɓ，一次性定 器到 后 止。

ƀ ь ь ƀ Ɔ Ƈ ƈ Đ Ɗ ƋƊ ƋƊ nanosleep 使用 完成相 休眠。clock_nanosleep đ ǫ TIMER_ABSTIME 表示 。休眠 如果 到未信号，moss 会 束本次等待，向用 返回 EINTR，并写回尚未用完的 。

3.3.5 子进程等待：登记后复查

父 程等待子 程状 ，可能在 完“没有可Ǝ 状 ”和登 Waker 之 ，子 程正好退出了。moss 的解 是登 后立 ：

```
match check_children().transpose() {
    Some(result) => Poll::Ready(result),  // 有状态，直接返回
    None => {
        child_exit_event.register(cx.waker());  // 登记 Waker
        match check_children().transpose() {     // 立即复查
            Some(result) => Poll::Ready(result),  // 补上窗口期的变化
            None => Poll::Pending,                // 确实没有，可以等了
        }
    }
}
```

代 3-5 登 后 ， 中 窗口

Ə Ɛ block_on 的思路一 ： 操作前后都 一 ， 保不 通知。

第二 已 Ƒ了子 程状 、等待 和接口 。子 程 用 exit 退出后，父 程通 wait 系列系ƀ 用回 。父 程需要ƒ 子 程（通 PID、 Ɠɣɦ ǀ Ǝɨ Ƙƌ ƀ

子 程状 化后，内核会 child_exit_event， 醒父 程。父 程恢 后重新 行 check_children，根据等待 断是 已有 果。如果 置了 WNOHANG， 使没有 果也立 返回； 等待直到有可∃ 的状 。

3.4 Futex 同步机制的改进

Futex 的改 最 ƙ， ☒要 理正 醒、超 、信号中断、位掩 、重新排 等多 。StarryOS 的等待 只保存了 (Waker, bitset)，分不清 。

3.4.1 独立通知状态

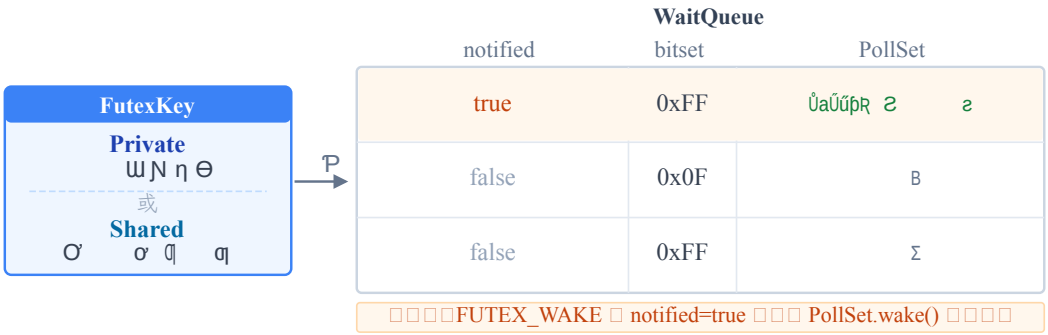
moss 把等待 ł 展成 ：(Arc<AtomicBool>, u32, Arc<PollSet>)。第一个 段 notified

```
// kernel/src/task/futex.rs
pub struct WaitQueue {
    // 每个等待项：(通知状态，位掩码，唤醒入口)
    queue: SpinNoIrq<VecDeque<(Arc<AtomicBool>, u32, Arc<PollSet>)>>,
}

pub fn wake(&self, count: usize, mask: u32) -> usize {
    let mut woke = 0;
    self.queue.lock().retain(|(waiter, bitset, poll)| {
        if woke >= count || (bitset & mask) == 0 {
            true  // 不唤醒，留在队列
        } else {
            waiter.store(true, Ordering::Release);  // 标记已通知
            poll.wake();                            // 唤醒任务
            woke += 1;
            false  // 从队列移除
        }
    });
    woke
}
```

代 3-6 Futex 等待 的 ƛ

有了独立的 notified 状 ，就能 断：超 是 没 醒， 是被 醒后 超 了。3-6 展示了 个 ƛ。



3-6 FutexKey 定位 列， 状

等待者只有在用 地址 等 眠。系 ƀ 用入口先 一次 ，不相等就返回 EAGAIN； 入 ，内核在持有等待 列 的 下再次 取用 。第二次 把用 断与入 操作 同一个 保 区 ，条件已 改 ，本次 用不再阻塞。

条件 成立 ，内核 建等待 并 入可中断、可超 的 Future。等待 程中会 notified Waker 前后都要 。 和 block_on 的 理思路一 ：通知可能在任何 到 ，用 把窗口 上。

3.4.2 精确清理

超 或信号中断 ，要把 个等待 从 列里 ſ 。moss 按 notified 的 象地址 配：

```
match result {
    Ok(Ok(())) => Ok(true),  // 正常唤醒
    Ok(Err(_)) | Err(_) => {
        // 超时或信号中断：删掉这一项
        let mut queue = self.queue.lock();
        queue.retain(|(waiter, ..)| !Arc::ptr_eq(waiter, &notified));
        Err(if result.is_ok() { AxError::TimedOut }
            else { AxError::Interrupted })
    }
}
```

代 3-7 按 象地址 ſ 除

Ż ļ ţ ſ T ſ Ż ļ Ţ Ư

正 事件之 ，信号也可以 束等待。interruptible 同 来的 Future 和任 的信号状 。 到未 信号后，等待提前 束，上 ư 系 ƀ 用再 定返回 EINTR、保存 或按信号 ʊ 重 U。

等待 超 或信号提前 束后， Ƴ ſ 除本 登 。 后 事件 可能 到旧 Waker，ƴ生 效 醒，也会 ƶ 醒数量 ƀ 。moss 用 Arc::ptr_eq 按 象地址 配， 保只 ſ 除当前 个等待 。

3.4.3 位掩码唤醒

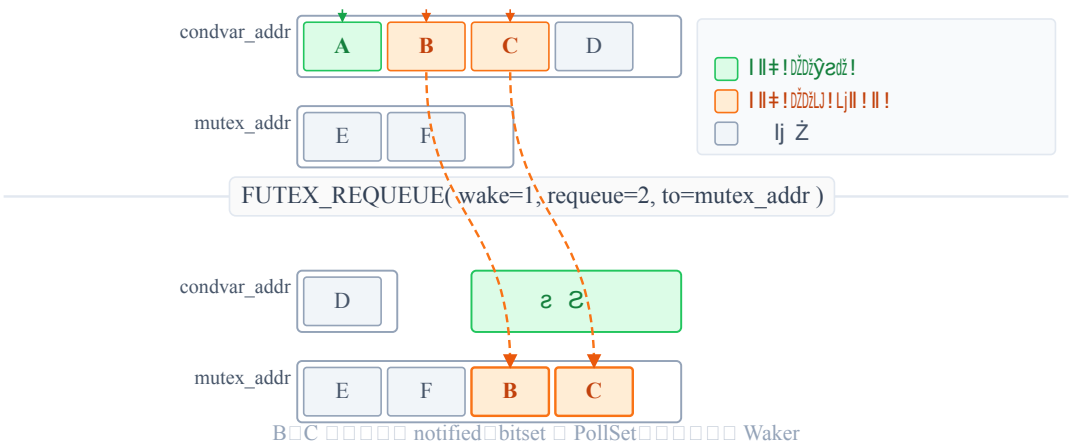
位掩 醒用来 ƶ 化条件 量等 。FUTEX_WAIT_BITSET Ʒ Ɛ ɛ FUTEX_WAKE_BITSET 只 醒 bitset 与 mask 有交 的任 ，从 避免 醒 程。

例如一个 Futex 上可能有多个不同的等待者：有的在等 事件（bitset = 0x01），有的在等写事件（bitset = 0x02）。 ƀ mask = 0x01 ʒ Ƨ ƽ ʂ Ư ƚ

wake 数 ƿ 列 会 每个等待 的 bitset：(bitset & mask) == 0 的 ，有交 的 醒。被 醒的等待 从 列移除，不会被重 理。

3.4.4 重新排队时保留状态

FUTEX_REQUEUE 把一 等待者从一个 Futex 移到另一个。moss 在移 保留完整的 ，不重新登 Waker。



3-7 重新排 整 移，保留所有状

Ż ļ NJ Ţ Ż ļ

Njnj ž pthread 条件 量。 不 把所有 程同 醒争 量，可以先醒 量 程，再把其 等待者 移到 量 的 列。 有 10 个 程在条件 量上等待，只 醒 1 个，其 9 个 移到 量 列。第一个 程 得 量后 理完 Ă ，量 列中的下一个 程再被 醒。 避免了 10 个 程同 被 醒后争 同一个 的 。

FUTEX_CMP_REQUEUE 在移 前 会 ă 地址的用 ，不 配 返回 EAGAIN。 是 了止条件 量和 量状 不一 的 ţ 移 。

3.4.5 时间语义修正

来的 没分清 FUTEX_WAIT 和 FUTEX_WAIT_BITSET 的 数。moss 按 Linux ʊ ƈ ь Ɗ ə FUTEX_CLOCK_REALTIME Nj Ǐ 相 需要加到当前 上得到截止 ， 本 就是截止 。如果截止 已了，等待立 束返回 ETIMEDOUT。 只向前 ，不 系 ƀ 整 ， 表示“再等待多 ”； 系 ƀ p ， 定一个 。

Ŭ ǐ Ŏ ŏ ż Ŭ pthread_cond_timedwait 使用 ，如果被当成相 理，超 就会 。

3.4.6 robust list 处理

lj ŭ Ű T f ū Ů Ż moss 完 了退出清理： p 程的 robust list， 到 的 Futex， owner_dead 并 醒一个等待者。被 醒的 程 到 EOWNERDEAD 后，由用 程 ů 并恢 的一 性。

```
// 线程退出时清理 robust list
for entry in robust_list.iter() {
    let futex_addr = entry.futex_offset;
    let futex = find_futex(futex_addr);

    // 标记持有者已死亡
    futex.owner_dead.store(true, Ordering::Release);
```

```
    // 唤醒一个等待者，让它知道锁的持有者已死
    futex.wq.wake(1, u32::MAX);
}

// 清零 clear_child_tid，通知等待该线程的其他线程
if let Some(tid_addr) = clear_child_tid {
    let futex = find_futex(tid_addr);
    futex.wq.wake(1, u32::MAX);
}
```

代 3-8 Futex 清理

Ů ǔ p Ů ROBUST_LIST_LIMIT 制，ù 的用 Ů表不会 内核 。 程退出 会清零 clear_child_tid，并 醒 个地址 的 Futex。 ə lj pthread_join 等 程回操作。

3.5 本章小结

StarryOS 已 有了基本的 度和事件等待，moss 的改 中在通知与阻塞交 的状理：

- **PollSet**： 醒，避免 度路 被 保
- **block_on**：woke
- ：(deadline, seq) 重， 旧事件
- ：登 后 ， 中 窗口
- **Futex**：独立 notified 状 + 「除＋状 保留

Ă ă 。通知先 再 醒，阻塞先 再改状 ，退出 按 清理。☒Ă修 了 LoongArch64 上 futex_wait、pthread cancellation 和 分休眠 ă Æ 的 。